

Algorithmique et Programmation 1

A. Boussidan, L. Exibard, P. Yvonnet

28 Septembre 2026

- C'est la fin du cycle « briques de base » !
- Les notes de cours sont disponibles sur E-learning.
- Pas de CC0...

- C'est la fin du cycle « briques de base » !
- Les notes de cours sont disponibles sur E-learning.
- Pas de CC0...
- ... Mais un TP noté la semaine prochaine !

Bref retour sur les booléens

Retour sur les booléens : tables de vérité

pluie	soleil	pluie or not soleil
False	False	
False	True	
True	False	
True	True	

Table de vérité de pluie or not soleil

pluie	soleil	pluie or not soleil == True
False	False	
False	True	
True	False	
True	True	

Table de vérité de pluie or not soleil == True

Retour sur les booléens : tables de vérité

pluie	soleil	pluie <code>or not</code> soleil
False	False	True
False	True	False
True	False	True
True	True	True

Table de vérité de pluie `or not` soleil

pluie	soleil	pluie <code>or not</code> soleil == True
False	False	
False	True	
True	False	
True	True	

Table de vérité de pluie `or not` soleil == True

C'est la même !

Retour sur les booléens : tables de vérité

pluie	soleil	pluie <code>or not</code> soleil
False	False	True
False	True	False
True	False	True
True	True	True

Table de vérité de pluie `or not` soleil

pluie	soleil	pluie <code>or not</code> soleil == True
False	False	True
False	True	False
True	False	True
True	True	True

Table de vérité de pluie `or not` soleil == True

C'est la même !

Retour sur les booléens : tables de vérité

p	q	p or q
False	False	False
False	True	True
True	False	True
True	True	True

Table de vérité de p or q

p	q	p and q
False	False	False
False	True	False
True	False	False
True	True	True

Table de vérité de p and q

p	not p
False	True
True	False

Table de vérité de not p

Évaluation séquentielle (ou paresseuse)

- $p \text{ or } q$ est vrai *dès que* p est vrai
 - $p \text{ and } q$ est faux *dès que* p est faux
- Pas besoin de regarder ce qu'il y a à droite !

Évaluation séquentielle (ou paresseuse)

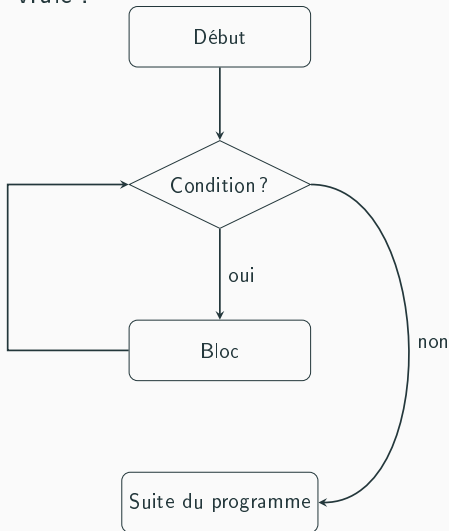
- p **or** q est vrai *dès que* p est vrai
 - p **and** q est faux *dès que* p est faux
- Pas besoin de regarder ce qu'il y a à droite !
- Python a la flemme

```
1/0 == 1                # Erreur !  
1 == 1 or 1/0 == 1     # Pas d'erreur !  
1 == 0 and 1/0 == 1    # Pas d'erreur !
```

Un peu plus sur les boucles `while`

Un retour sur les boucles `while`

Que faire si on veut répéter une instruction si une condition reste vraie ?



```
# début
while condition:
    # bloc v
    # (corps de la boucle)
# suite
```

Itération

Une exécution complète du corps de la boucle.

Un peu de vocabulaire

Itération

Une exécution complète du corps de la boucle.

Variable ou indice de boucle

La variable qui compte le nombre de tours de boucles (si elle existe).

→ Souvent notée i ou j .

Un peu de vocabulaire

Itération

Une exécution complète du corps de la boucle.

Variable ou indice de boucle

La variable qui compte le nombre de tours de boucles (si elle existe).

→ Souvent notée i ou j .

Condition de continuation

La condition qui est vérifiée à chaque début de boucle : si elle est vraie, on continue.

Un peu de vocabulaire

Itération

Une exécution complète du corps de la boucle.

Variable ou indice de boucle

La variable qui compte le nombre de tours de boucles (si elle existe).

→ Souvent notée i ou j .

Condition de continuation

La condition qui est vérifiée à chaque début de boucle : si elle est vraie, on continue.

Condition d'arrêt

La négation de la condition de continuation.

Sur un exemple

```
i = 0 # 1
while i <= 15: # 2
    print("Ceci est l'itération numéro", i )
    i = i + 1 # 3
#4
```



Images/QR_code_wooclap.png

Sur un exemple

```
i = 0 # indice de boucle
while i <= 15: #condition de continuation
    print("Ceci est l'itération numéro", i )
    i = i + 1 #Incrémentation
# Ici, i > 15: condition d'arrêt
```

Un exemple familier...

```
from random import randint

win_L, win_T = 0, 0
courses = 0  # indice de boucle 1

while courses < 1000:  # condition de continuation (boucle 1)
    pos_L = 0
    pos_T = 0
    winner = "Tortue"
    while pos_T < 6 and pos_L < 6:  # condition de continuation (boucle 2)
        lancer = randint(1, 6)
        if lancer == 6:
            pos_L = 6
            win_L += 1
        else:
            pos_T += lancer
            if pos_T >= 6:
                win_T += 1
```

Un exemple familier...

```
from random import randint

win_L, win_T = 0, 0
courses = 0  # indice de boucle 1

while courses < 1000:  # condition de continuation (boucle 1)
    pos_L = 0
    pos_T = 0
    winner = "Tortue"
    while pos_T < 6 and pos_L < 6:  # condition de continuation (boucle 2)
        lancer = randint(1, 6)
        if lancer == 6:
            pos_L = 6
            win_L += 1
        else:
            pos_T += lancer
            if pos_T >= 6:
                win_T += 1
```

M'sieur, ça marche pas !

Un exemple familial... corrigé

```
from random import randint

win_L, win_T = 0, 0
courses = 0 # indice de boucle 1

while courses < 1000: # condition de continuation (boucle 1)
    pos_L = 0
    pos_T = 0
    winner = "Tortue"
    while pos_T < 6 and pos_L < 6: # condition de continuation (boucle 2)
        lancer = randint(1, 6)
        if lancer == 6:
            pos_L = 6
            win_L += 1
        else:
            pos_T += lancer
            if pos_T >= 6:
                win_T += 1
    courses += 1 # Ne pas oublier la mise à jour !
```

Attention à la boucle infinie

Comment débogguer une boucle while

- Ajouter des `print` dans son code

Comment débogguer une boucle while

- Ajouter des `print` dans son code
- Exécuter son code pas à pas (à la main, sur PythonTutor, ou avec le débogueur)

Comment déboguer une boucle while

- Ajouter des `print` dans son code
- Exécuter son code pas à pas (à la main, sur PythonTutor, ou avec le débogueur)
- S'il y a une boucle infinie : se concentrer sur les variables du test de continuation
 - Sont-elles bien initialisées ?
 - Quand sont-elles mises à jour ?
 - Pourquoi est-ce que la boucle finit par s'arrêter ?

Boucles while - Cas d'utilisation

Rappel - être un chef!

On commence toujours par formuler l'algorithme. Si votre algorithme contient ...

Boucles while - Cas d'utilisation

Rappel - être un chef!

On commence toujours par formuler l'algorithme. Si votre algorithme contient ...

"...et on recommence"

"jusqu'à ce que..."

"s'il en reste, on refait"

"on repart du début"

Boucles while - Cas d'utilisation

Rappel - être un chef!

On commence toujours par formuler l'algorithme. Si votre algorithme contient ...

"...et on recommence"

"jusqu'à ce que..."

"s'il en reste, on refait"

"on repart du début"

Il faut une boucle while !

Exemple - saisie contrôlée

Principe de la saisie contrôlée

Demander à l'utilisateur de saisir quelque chose et vérifier que sa saisie est correcte

Quel est l'algorithme ?

Exemple - saisie contrôlée

Principe de la saisie contrôlée

Demander à l'utilisateur de saisir quelque chose et vérifier que sa saisie est correcte

Quel est l'algorithme ?

"On demande de saisir une première fois,
et on recommence tant que la saisie est incorrecte."

Exemple - saisie contrôlée

Principe de la saisie contrôlée

Demander à l'utilisateur de saisir quelque chose et vérifier que sa saisie est correcte

Quel est l'algorithme ?

"**On demande** de saisir **une première fois**,
et **on recommence tant que** la saisie est
incorrecte."

Boucle while



Exemple - saisie contrôlée

Principe de la saisie contrôlée

Demander à l'utilisateur de saisir quelque chose et vérifier que sa saisie est correcte

Quel est l'algorithme ?

input

"**On demande** de saisir **une première fois**,
et **on recommence tant que** la saisie est
incorrecte."

Boucle while

Exemple - saisie contrôlée

Principe de la saisie contrôlée

Demander à l'utilisateur de saisir quelque chose et vérifier que sa saisie est correcte

Quel est l'algorithme ?

input

initialisation
avant la boucle

"**On demande** de saisir **une première fois**,
et **on recommence tant que** la saisie est
incorrecte."

Boucle while

Saisie contrôlée - Une implémentation

```
parfum = input("Choisissez un parfum de glace")

while parfum != "chocolat" and parfum != "vanille" and parfum != "fraise":
    print("Nous n'avons pas ça en stock...")
    parfum = input("Choisissez un parfum de glace")

print("Bien sûr, voici votre glace parfum " + parfum + " !")
```

Saisie contrôlée - Une implémentation

```
parfum = input("Choisissez un parfum de glace")

while parfum != "chocolat" and parfum != "vanille" and parfum != "fraise":
    print("Nous n'avons pas ça en stock...")
    parfum = input("Choisissez un parfum de glace")

print("Bien sûr, voici votre glace parfum " + parfum + " !")
```

- Comment faire pour ajouter d'autres parfums possibles ?
- Que se passe-t-il si on remplace les `and` par des `or` ?
- On est obligé de copier/coller la ligne avec le `input`

Un nouveau problème

On sait déjà répéter une instruction avec un `while`.

Comment afficher chaque lettre d'un mot, une par une?

Un nouveau problème

On sait déjà répéter une instruction avec un `while`.

Comment afficher chaque lettre d'un mot, une par une?

```
mot = "Python"
```

Indice ou élément ?

P	y	t	h	o	n
---	---	---	---	---	---

Indice ou élément ?

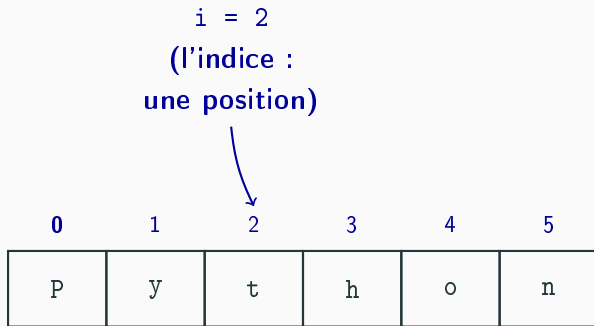
0	1	2	3	4	5
P	y	t	h	o	n

Indice ou élément ?

0	1	2	3	4	5
P	y	t	h	o	n

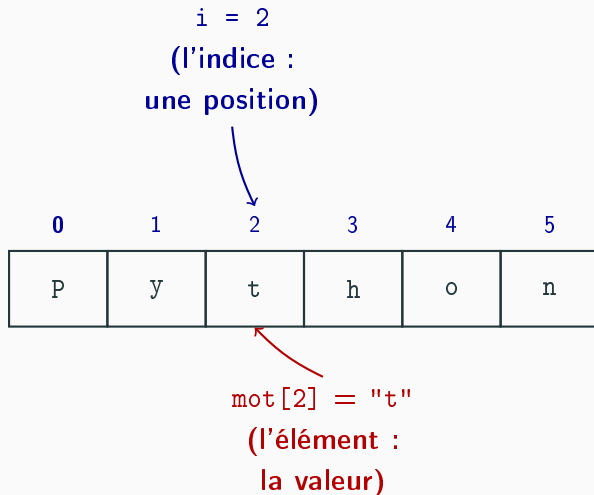
Dans une `str`, les éléments sont numérotés à partir de 0.

Indice ou élément ?



Dans une `str`, les éléments sont numérotés à partir de 0.

Indice ou élément ?



Dans une `str`, les éléments sont numérotés à partir de 0.

Quel **algorithme** utiliser pour afficher les lettres une par une ?

Quel **algorithme** utiliser pour afficher les lettres une par une ?

On va faire parcourir les **positions** de la chaîne, et on va afficher
l'**élément** à cette position.

Quel **algorithme** utiliser pour afficher les lettres une par une ?

On va faire parcourir les **positions** de la chaîne, et on va afficher
l'**élément** à cette position.

```
mot = "Python"
position = 0
while position < len(mot):
    print(mot[position])
    position = position + 1
```

Quel **algorithme** utiliser pour afficher les lettres une par une ?

On va faire parcourir les **positions** de la chaîne, et on va afficher
l'**élément** à cette position.

```
mot = "Python"
i = 0
while i < len(mot):
    print(mot[i])
    i = i + 1
```

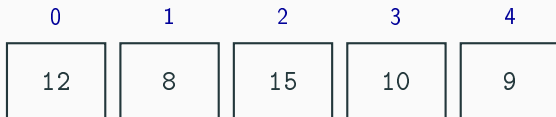
Quel **algorithme** utiliser pour afficher les lettres une par une ?

On va faire parcourir les **positions** de la chaîne, et on va afficher
l'**élément** à cette position.

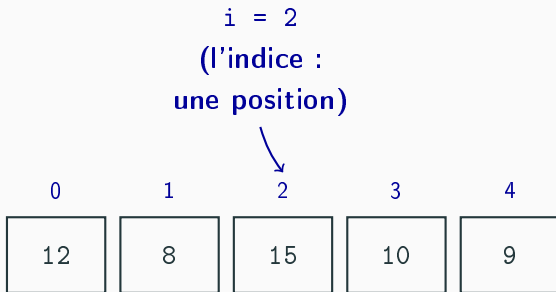
```
mot = "Python"
i = 0
while i < len(mot):
    print(mot[i])
    i = i + 1
```

- i varie de 0 à $\text{len}(\text{mot}) - 1$
- à chaque tour, on accède à `mot[i]`

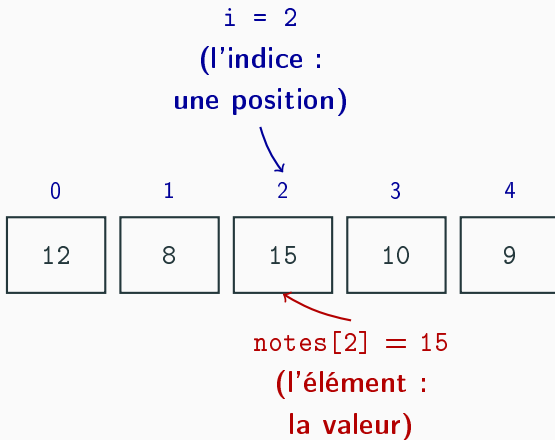
Et pour une liste ?



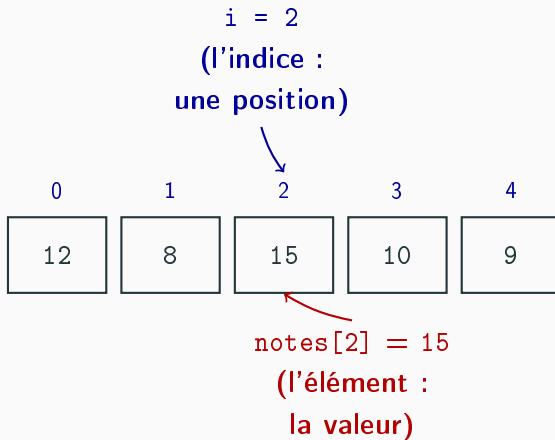
Et pour une liste ?



Et pour une liste ?



Et pour une liste ?



Ca devrait être familier...

Et pour une liste ?

Le même programme marche pour une liste !

```
notes = [12, 8, 15, 10]
i = 0
while i < len(notes):
    print(notes[i])
    i = i + 1
```

Et pour une liste ?

Le même programme marche pour une liste !

```
notes = [12, 8, 15, 10]
i = 0
while i < len(notes):
    print(notes[i])
    i = i + 1
```

Coïncidence ? ...

Et pour une liste ?

Le même programme marche pour une liste !

```
notes = [12, 8, 15, 10]
i = 0
while i < len(notes):
    print(notes[i])
    i = i + 1
```

Coïncidence ? ...

Non, itérables !

Itérables et boucle `for`

Un **itérable** est un objet qui regroupe plusieurs valeurs, que l'on peut parcourir élément par élément.

En Python, les itérables les plus courants sont :

- `str`
- `list`
- `tuple` (vu plus tard)
- `dict` (vu plus tard)

Un **itérable** est un objet qui regroupe plusieurs valeurs, que l'on peut parcourir élément par élément.

En Python, les itérables les plus courants sont :

- `str`
- `list`
- `tuple` (vu plus tard)
- `dict` (vu plus tard)
- `range` (vu dans quelques instants)

Un outil pour parcourir les itérables : la boucle `for` !

Un outil pour parcourir les itérables : la boucle `for` !

À retenir

La boucle `for` parcourt toujours les **éléments** d'un itérable.

Un outil pour parcourir les itérables : la boucle for !

À retenir

La boucle for parcourt toujours les éléments d'un itérable.

str

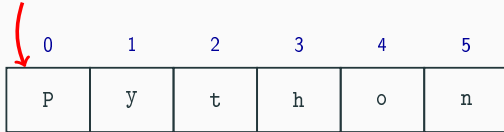
```
mot = "Python"  
for element in mot:  
    print(element)
```

list

```
notes = [12, 8, 15, 10, 9]  
for element in notes:  
    print(element)
```

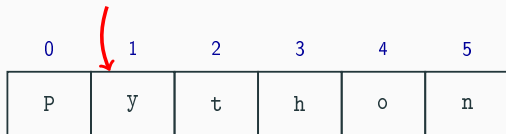
La boucle for sur une str

> P



```
mot = "Python"  
for element in mot:  
    print(element)
```

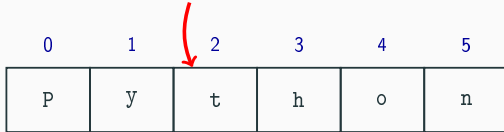
La boucle for sur une str



```
> P  
> y
```

```
mot = "Python"  
for element in mot:  
    print(element)
```

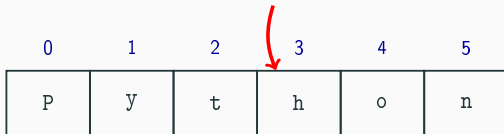
La boucle for sur une str



```
> P  
> y  
> t
```

```
mot = "Python"  
for element in mot:  
    print(element)
```

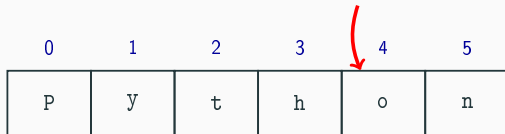
La boucle for sur une str



```
> P  
> y  
> t  
> h
```

```
mot = "Python"  
for element in mot:  
    print(element)
```

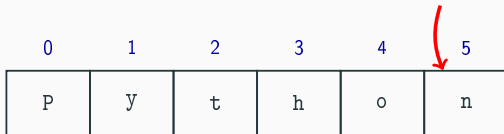
La boucle for sur une str



```
> P  
> y  
> t  
> h  
> o
```

```
mot = "Python"  
for element in mot:  
    print(element)
```

La boucle for sur une str



```
> P  
> y  
> t  
> h  
> o  
> n
```

```
mot = "Python"  
for element in mot:  
    print(element)
```

La boucle for sur une list



0	1	2	3	4
12	8	15	10	9

> 12

```
notes = [12, 8, 15, 10, 9]
for element in notes:
    print(element)
```

La boucle for sur une list



0	1	2	3	4
12	8	15	10	9

```
> 12  
> 8
```

```
notes = [12, 8, 15, 10, 9]  
for element in notes:  
    print(element)
```

La boucle for sur une list

0	1	2	3	4
12	8	15	10	9

> 12
> 8
> 15

```
notes = [12, 8, 15, 10, 9]
for element in notes:
    print(element)
```

La boucle for sur une list

0	1	2	3	4
12	8	15	10	9

> 12
> 8
> 15
> 10

```
notes = [12, 8, 15, 10, 9]
for element in notes:
    print(element)
```

La boucle for sur une list

0	1	2	3	4
12	8	15	10	9

```
notes = [12, 8, 15, 10, 9]
for element in notes:
    print(element)
```

```
> 12
> 8
> 15
> 10
> 9
```

La boucle for sur une list

0	1	2	3	4
12	8	15	10	9

```
notes = [12, 8, 15, 10, 9]
for element in notes:
    print(element)
```

```
> 12
> 8
> 15
> 10
> 9
```

Et `for i in range(5)` ?

```
range(a, b)
```

génère les entiers de a (inclus) à b (exclu)

```
range(a, b)
```

génère les entiers de a (inclus) à b (exclu)

```
range(0, 6)
```

L'itérable range

`range(a, b)`

génère les entiers de a (inclus) à b (exclu)

`range(0, 6)`

0	1	2	3	4	5
---	---	---	---	---	---

La boucle for avec range

```
for i in range(0, 6):  
    print(i)
```

La boucle for avec range

```
for i in range(0, 6):  
    print(i)
```

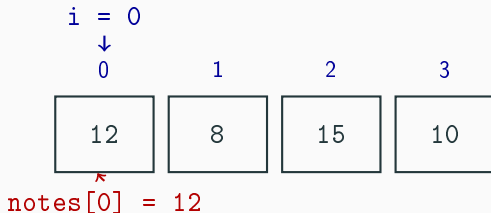
i prend successivement chaque **élément** de range(0, 6) :
0, 1, 2, 3, 4, 5.

```
> 0  
> 1  
> 2  
> 3  
> 4  
> 5
```

for i in range pour parcourir les indices

```
notes = [12, 8, 15, 10]
```

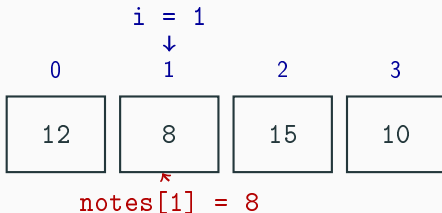
```
for i in range(4):  
    print("Position : ", i)  
    print("Element : ", notes[i])
```



for i in range pour parcourir les indices

```
notes = [12, 8, 15, 10]
```

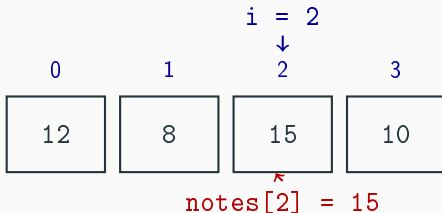
```
for i in range(4):  
    print("Position : ", i)  
    print("Element : ", notes[i])
```



for i in range pour parcourir les indices

```
notes = [12, 8, 15, 10]
```

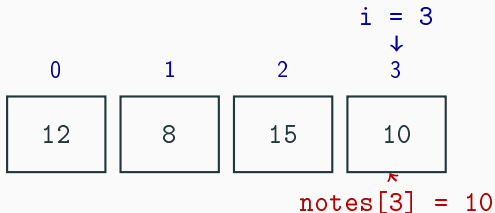
```
for i in range(4):  
    print("Position : ", i)  
    print("Element : ", notes[i])
```



for i in range pour parcourir les indices

```
notes = [12, 8, 15, 10]
```

```
for i in range(4):  
    print("Position : ", i)  
    print("Element : ", notes[i])
```



Retour sur les listes

Gérer un étudiant

```
nom = "Dupont"  
note = 14  
  
print(nom, note)
```

Facile ! Et si j'ai **2** étudiants à gérer ?

Et pour 2 étudiants ?

```
nom_1 = "Dupont"  
note_1 = 14  
nom_2 = "Martin"  
note_2 = 9  
  
print(nom_1, note_1)  
print(nom_2, note_2)
```

Et pour 2 étudiants ?

```
nom_1 = "Dupont"  
note_1 = 14  
nom_2 = "Martin"  
note_2 = 9  
  
print(nom_1, note_1)  
print(nom_2, note_2)
```

Et pour 5 ? Pour 30 ?

Et pour 5 ? Pour 30 ?

```
nom_1 = "Dupont"  
note_1 = 14  
nom_2 = "Martin"  
note_2 = 9  
nom_3 = "Durand"  
note_3 = 16  
nom_4 = "Petit"  
note_4 = 11  
nom_5 = "Bernard"  
note_5 = 13
```

Et pour 5 ? Pour 30 ?

```
nom_1 = "Dupont"  
note_1 = 14  
nom_2 = "Martin"  
note_2 = 9  
nom_3 = "Durand"  
note_3 = 16  
nom_4 = "Petit"  
note_4 = 11  
nom_5 = "Bernard"  
note_5 = 13
```

C'est un peu laborieux...

Et pour 5 ? Pour 30 ?

```
nom_1 = "Dupont"  
note_1 = 14  
nom_2 = "Martin"  
note_2 = 9  
nom_3 = "Durand"  
note_3 = 16  
nom_4 = "Petit"  
note_4 = 11  
nom_5 = "Bernard"  
note_5 = 13
```

C'est un peu laborieux...

Et si je veux gérer un nombre **non borné**
d'étudiants ?

Un bon cas d'utilisation d'une liste

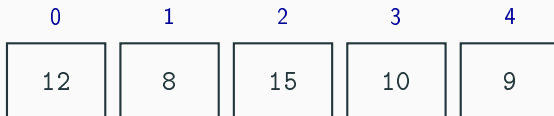
```
notes = [12, 8, 15, 10, 9]  
print(notes)
```

Une liste range **plusieurs valeurs** dans une **seule variable**, dans un ordre précis.

Si on doit dupliquer plusieurs variables, c'est un bon moment pour les remplacer par une liste !

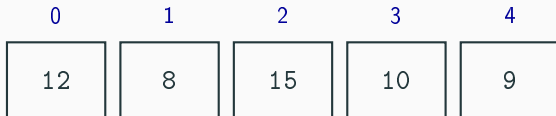
Accéder à un élément : l'indexation

```
notes = [12, 8, 15, 10, 9]  
print(notes[0])  
print(notes[2])
```



Accéder à un élément : l'indexation

```
notes = [12, 8, 15, 10, 9]  
print(notes[0])  
print(notes[2])
```



Attention

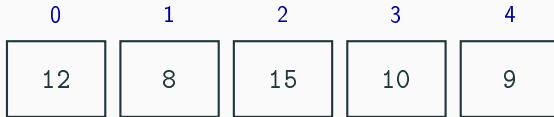
Le **premier** élément est à l'indice **0**, pas 1 !

Et le dernier élément ?

0	1	2	3	4
12	8	15	10	9

Pour une liste de **5** éléments, quel est l'indice du dernier ?

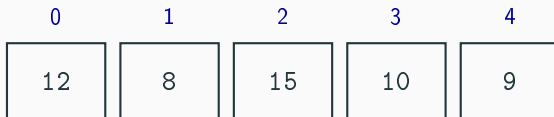
Et le dernier élément ?



Pour une liste de **5** éléments, quel est l'indice du dernier ?

`notes[4]` , pas `notes[5]` !

Et le dernier élément ?



Pour une liste de **5** éléments, quel est l'indice du dernier ?

`notes[4]` , pas `notes[5]` !

Une liste de n éléments a des indices de 0 à $n-1$.

Et le dernier élément ?



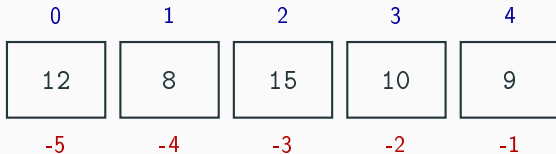
Pour une liste de **5** éléments, quel est l'indice du dernier ?

`notes[4]` , pas `notes[5]` !

Une liste de `n` éléments a des indices de 0 à `len(liste)-1`.

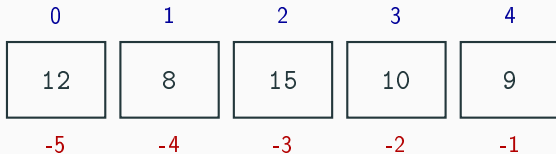
Index négatifs

```
notes = [12, 8, 15, 10, 9]  
print(notes[-1])  
print(notes[-2])
```



Index négatifs

```
notes = [12, 8, 15, 10, 9]
print(notes[-1])
print(notes[-2])
```



`notes[-1]` : le dernier élément, sans même connaître la longueur de la liste !

Modifier, ajouter, retirer un élément

`notes[0] = 13`

on remplace l'élément en position 0. Il y a maintenant 13 dans la première case de la liste.



Modifier, ajouter, retirer un élément

```
notes.append(17)
```

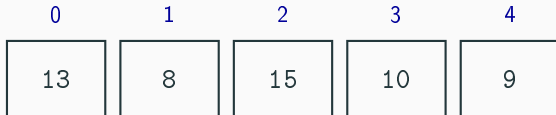
on ajoute 17 à la fin

0	1	2	3	4	5
13	8	15	10	9	17

Modifier, ajouter, retirer un élément

`notes.pop()`

`notes.pop()` retire (et renvoie) le **dernier** élément de la liste.



Attention aux erreurs

```
notes = [12, 8, 15, 10, 9]  
print(notes[10])
```

Attention aux erreurs

```
notes = [12, 8, 15, 10, 9]  
print(notes[10])
```

```
> IndexError: list index out of range
```

Attention aux erreurs

```
notes = [12, 8, 15, 10, 9]  
print(notes[10])
```

```
> IndexError: list index out of range
```

Attention

notes n'a que 5 éléments (indices 0 à 4) : l'indice 10 n'existe pas.

Attention aux erreurs

```
notes = [12, 8, 15, 10, 9]
i = 0
while i <= len(notes):
    print(notes[i])
    i = i + 1
```

Attention aux erreurs

```
notes = [12, 8, 15, 10, 9]
i = 0
while i <= len(notes):
    print(notes[i])
    i = i + 1
```

```
> 12
> 8
> 15
> 10
> 9
> IndexError: list index out of range
```

Vérifier qu'un élément est dans une liste

`element in liste` : vrai si l'élément est présent

`element not in liste` : vrai si l'élément est absent

Vérifier qu'un élément est dans une liste

`element in liste` : vrai si l'élément est présent

`element not in liste` : vrai si l'élément est absent

```
parfum = input("Choisissez un parfum de glace")

while parfum != "chocolat" and parfum != "vanille" and parfum != "fraise":
    print("Nous n'avons pas ça en stock...")
    parfum = input("Choisissez un parfum de glace")

print("Bien sûr, voici votre glace parfum " + parfum + " !")
```

- **Comment faire pour ajouter d'autres parfums possibles ?**
- Que se passe-t-il si on remplace les `and` par des `or` ?
- On est obligé de copier/coller la ligne avec le `input`

Vérifier qu'un élément est dans une liste

`element in liste` : vrai si l'élément est présent

`element not in liste` : vrai si l'élément est absent

```
parfums = ["chocolat", "vanille", "fraise"]
parfum = input("Choisissez un parfum de glace")

while parfum not in parfums:
    print("Nous n'avons pas ça en stock...")
    parfum = input("Choisissez un parfum de glace")

print("Bien sûr, voici votre glace parfum " + parfum + " !")
```

On utilise une liste de parfums, et `not in` pour tester l'appartenance.

Les fonctions

Pas facile à lire !

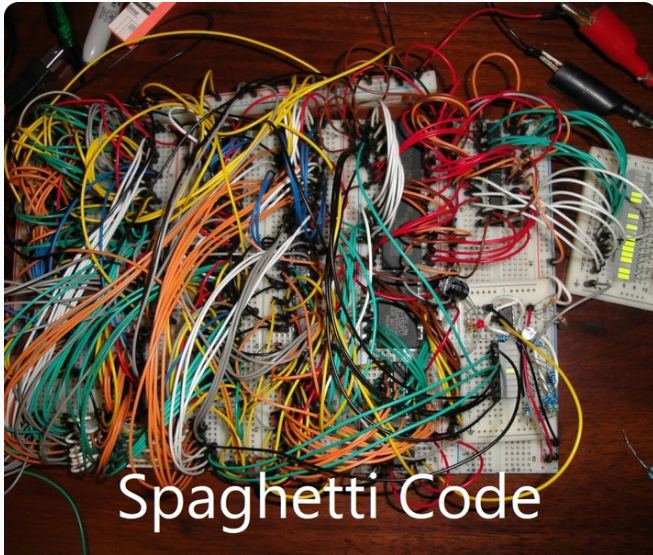
```
i = 0
while i < n:
    j = 0
    while j < n:
        print(caractere, end = '')
        j += 1
    print('\n', end = '')
    i += 1
```

Une fonction : qu'est-ce que c'est ?

En informatique, une fonction est :

- Un morceau de programme
 - Portant en général *un nom*
 - Prenant un ou plusieurs *paramètres* (ou zéro)
 - Renvoyant un résultat (la plupart du temps)
- Ça ressemble beaucoup aux fonctions vues en mathématiques !

Une fonction : à quoi ça sert ?



Spaghetti Code

Une fonction : à quoi ça sert ?

Lisibilité

- Isoler les parties du programme selon ce qu'elles font
- Éviter trop d'imbrications entre `if` et `while`

Une fonction : à quoi ça sert ?

Lisibilité

- Isoler les parties du programme selon ce qu'elles font
- Éviter trop d'imbrications entre `if` et `while`

Modularité et robustesse

- Réutiliser le code plusieurs fois
- Faciliter la correction des bugs

Une fonction : à quoi ça sert ?

Lisibilité

- Isoler les parties du programme selon ce qu'elles font
- Éviter trop d'imbrications entre `if` et `while`

Modularité et robustesse

- Réutiliser le code plusieurs fois
- Faciliter la correction des bugs

Généricité

- Changer la valeur des paramètres

Une fonction : qu'est-ce que c'est ?



Fonctions prédéfinies

→ On en a déjà vu !

Exemples

Fonctions prédéfinies

→ On en a déjà vu !

Exemples

- `print(s)` : afficher la chaîne de caractères `s`
- `input(s)` : afficher la chaîne de caractères `s` et attendre une entrée utilisateur
- `int(x)` : convertit l'objet `x` en entier
- `len(x)` : renvoie la longueur de l'objet `x`

Fonctions prédéfinies

→ On en a déjà vu !

Exemples

- `print(s)` : afficher la chaîne de caractères `s`
- `input(s)` : afficher la chaîne de caractères `s` et attendre une entrée utilisateur
- `int(x)` : convertit l'objet `x` en entier
- `len(x)` : renvoie la longueur de l'objet `x`

Modules

Bibliothèque de fonctions, de types et d'objets

Exemples

Fonctions prédéfinies

→ On en a déjà vu !

Exemples

- `print(s)` : afficher la chaîne de caractères `s`
- `input(s)` : afficher la chaîne de caractères `s` et attendre une entrée utilisateur
- `int(x)` : convertit l'objet `x` en entier
- `len(x)` : renvoie la longueur de l'objet `x`

Modules

Bibliothèque de fonctions, de types et d'objets

Exemples

- `randint(a,b)` (module `randint`)
- renvoie un entier aléatoire compris entre `a` et `b`

Définition et appel de fonction

Définir une fonction

La syntaxe pour définir une fonction est :

```
def nom_fonction(parametre_1, ..., parametre_n):  
    # corps de la fonction  
    # utilisant parametre_1, ..., parametre_n  
    ...  
    # peut renvoyer un résultat :  
    return resultat
```

Définition et appel de fonction

Définir une fonction

La syntaxe pour définir une fonction est :

```
def nom_fonction(parametre_1, ..., parametre_n):  
    # corps de la fonction  
    # utilisant parametre_1, ..., parametre_n  
    ...  
    # peut renvoyer un résultat :  
    return resultat
```

Appeler une fonction

On peut ensuite *appeler* la fonction `nom_fonction` dans le code :

```
resultat = nom_fonction(expression_1, ..., expression_n)
```

```
def nom_fonction(parametre_1, ..., parametre_n):  
    # corps de la fonction  
    # utilisant parametre_1, ..., parametre_n  
    ...  
    # peut renvoyer un résultat :  
    return resultat
```

- Type des arguments (s'il y en a)
- Type de la valeur de retour

Fonctions : exemples

Fonction à paramètres et résultats

Calculer le maximum de deux entiers :

Fonction à paramètres et résultats

Calculer le maximum de deux entiers :

```
def maximum(a,b):  
    if a >= b:  
        return a  
    else:  
        return b
```

Fonction à paramètres et résultats

Calculer le maximum de deux entiers :

```
def maximum(a,b):  
    if a >= b:  
        return a  
    else:  
        return b
```

Entraînement

- Ouvrir le programme maximum.py
- Décrire l'exécution pas à pas du programme (avec état de la mémoire)
- On peut aussi essayer avec Python Tutor
- Dresser un tableau de valeurs de l'exécution du programme

Fonction sans paramètres

Simuler un lancé de dés, et compter combien de coups il faut pour faire un 6.

Fonction sans paramètres

Simuler un lancé de dés, et compter combien de coups il faut pour faire un 6.

```
from random import randint
def lance_de():
    return randint(1,6)

compteur = 1
while lance_de() != 6:
    compteur = compteur + 1
print('Obtenu un 6 en', compteur, 'jets de dé.')
```

Fonction sans paramètres

Simuler un lancé de dés, et compter combien de coups il faut pour faire un 6.

```
from random import randint
def lance_de():
    return randint(1,6)

compteur = 1
while lance_de() != 6:
    compteur = compteur + 1
print('Obtenu un 6 en', compteur, 'jets de dé.')
```

Ce que ça donne sur Python Tutor.

Fonction sans valeur de retour

Dessiner un carré fait du caractère caractère :

Fonction sans valeur de retour

Dessiner un carré fait du caractère caractere :

```
def dessine_carre(n, caractere):  
    i = 0  
    while i < n:  
        j = 0  
        while j < n:  
            print(caractere, end = ' ')  
            j += 1  
        print('\n', end = ' ')  
        i += 1  
    return
```

Fonction sans valeur de retour

Dessiner un carré fait du caractère caractere :

```
def dessine_carre(n, caractere):  
    i = 0  
    while i < n:  
        j = 0  
        while j < n:  
            print(caractere, end = ' ')  
            j += 1  
        print('\n', end = ' ')  
        i += 1  
    return
```

Ce que ça donne sur Python Tutor.

Paramètre / saisie

```
def maximum(a,b):  
    a = int(input()) # NON !  
    b = int(input()) # NON !  
    if a >= b:  
        return a  
    else:  
        return b
```

Paramètre / saisie

```
def maximum(a,b):  
    a = int(input()) # NON !  
    b = int(input()) # NON !  
    if a >= b:  
        return a  
    else:  
        return b
```

Retour / affichage

```
def maximum(a,b):  
    if a >= b:  
        print(a) # NON !  
    else:  
        print(b) # NON !
```

On peut appeler une fonction dans une fonction ! (et ainsi de suite)

Dessiner un carré : variante

```
def dessine_ligne(n, caractere):  
    j = 0  
    while j < n:  
        print(caractere, end = '')  
        j += 1  
    print('\n', end = '')  
    return
```

```
def dessine_carre(n, caractere):  
    i = 0  
    while i < n:  
        dessine_ligne(n,caractere)  
        i += 1  
    return
```

Fonctions et espaces de nom

- les paramètres et variables définies dans le corps d'une fonction sont **indépendantes** des autres variables du programme
- elles n'existent plus une fois l'exécution de la fonction terminée
- on les appelle des variables *locales*

Par conséquent, on peut renommer les variables d'une fonction.

→ Démonstration sur Thonny (`maximum.py`)

(Contre-)exemple : intervertir des variables

- Dans `maximum.py`, changer les valeurs de `a` et `b` dans la fonction n'a pas d'effet sur `x` et `y` dans le programme principal!
- Dans `interversion.py`, la variable `temp` n'existe plus après l'exécution de la fonction